

The mathematics behind WARPDRIVES.SKYRENDER

What you see through the windshield of an Alcubierre bubble
and how the renderer computes it

WarpDrives contributors

Scope. This note documents the physics, geometry, and numerical scheme behind the `warpbubblesim.viz.skyrender*` pipeline. Every formula here maps to a function in the Python (or JAX) renderer; cross-references are given at the end of each section.

Contents

1 Setup: warp metrics in ADM form	1
2 Geodesics: how light travels through the bubble	2
2.1 Computing $\partial_\alpha g_{\mu\nu}$	2
3 The observer’s frame: orthonormal tetrads	3
4 Camera model: pixels to past-pointing null tangents	3
5 Mapping the asymptotic direction to the celestial sphere	4
6 Honest physics: Doppler brightness and front horizon	4
6.1 Doppler factor from Killing energy	4
6.2 Brightness scaling: Liouville’s invariant	5
6.3 Front horizon as a black region	5
7 Numerical scheme summary	5
8 Velocity schedule	6
9 Conventions cheatsheet	6
10 A common misconception	6

1 Setup: warp metrics in ADM form

We work in geometric units ($G = c = 1$) with metric signature $(-, +, +, +)$ and coordinates (t, x, y, z) , indexed as x^μ with $\mu \in \{0, 1, 2, 3\}$.

A warp drive metric is given in 3+1 (ADM) form by a lapse $\alpha(t, \mathbf{x})$, a shift $\beta^i(t, \mathbf{x})$, and a spatial metric $\gamma_{ij}(t, \mathbf{x})$:

$$ds^2 = -\alpha^2 dt^2 + \gamma_{ij}(dx^i + \beta^i dt)(dx^j + \beta^j dt). \quad (1)$$

Equivalently, the 4-metric components are

$$g_{tt} = -\alpha^2 + \beta_i \beta^i, \quad g_{ti} = \beta_i, \quad g_{ij} = \gamma_{ij}, \quad (2)$$

with $\beta_i \equiv \gamma_{ij}\beta^j$. All warp metrics in `warpbubblesim` have $\alpha = 1$ and $\gamma_{ij} = \delta_{ij}$, i.e. “flat slices, unit lapse, all the warp lives in the shift.”

The bubble centre travels at coordinate velocity $v_s \equiv dx_s/dt$ along the x -axis, with $x_s(t) = x_0 + v_s t$. The radial coordinate from the bubble centre is

$$r_s(t, x, y, z) = \sqrt{(x - x_s(t))^2 + y^2 + z^2}. \quad (3)$$

A smooth shape function $f(r)$ tapers the warp from $f(0) = 1$ inside the bubble to $f(\infty) = 0$ outside.

Alcubierre (1994)

The shift is purely along x :

$$\beta_{\text{Alc}}^i = (-v_s f(r_s), 0, 0). \quad (4)$$

Substituting into (2) gives the explicit form

$$ds_{\text{Alc}}^2 = -dt^2 + (dx - v_s f(r_s) dt)^2 + dy^2 + dz^2. \quad (5)$$

At the bubble centre $f = 1$, so $g_{tt} = -1 + v_s^2$, $g_{tx} = -v_s$, $g_{ii} = 1$ and the determinant is $\det g = -1$ for all v_s . The crucial property that makes Alcubierre superluminally consistent is that the worldline $x = x_s(t)$ has tangent $u^\mu = (1, v_s, 0, 0)$ and

$$g_{\mu\nu}u^\mu u^\nu = -1 + v_s^2 + 2(-v_s)v_s + v_s^2 = -1 \quad (6)$$

for all v_s — the bubble-centre observer is unit-timelike (proper time matches coordinate time) at any speed.

Natário (2002)

Natário’s expansion-free shift is divergence-free: $\nabla \cdot \beta = 0$. Using the vector potential $\mathbf{A} = v_s g(r_s) (0, z, -y)$ and $\beta = \nabla \times \mathbf{A}$ gives, with envelope $g(r)$ and $g' \equiv dg/dr$,

$$\begin{aligned} \beta_{\text{Nat}}^x &= -v_s \left(2g(r_s) + \frac{y^2 + z^2}{r_s} g'(r_s) \right), \\ \beta_{\text{Nat}}^y &= +v_s \frac{y(x - x_s)}{r_s} g'(r_s), \\ \beta_{\text{Nat}}^z &= +v_s \frac{z(x - x_s)}{r_s} g'(r_s). \end{aligned} \quad (7)$$

At $r_s \rightarrow 0$ the off-axis components vanish and $\beta^x \rightarrow -2v_s g(0)$. Critically, the bubble-centre worldline $u = (1, v_s, 0, 0)$ now has

$$g_{\mu\nu}u^\mu u^\nu = -(1 - v_s^2), \quad (8)$$

which is timelike only for $|v_s| < 1$. Natário’s central observer *cannot* be superluminal; the renderer caps the comparison sweep at $v_s = 0.95$ for that reason.

Shape function (default: tanh)

$$f(r) = \frac{\tanh \sigma(r + R) - \tanh \sigma(r - R)}{2 \tanh(\sigma R)}, \quad f(0) = 1, \quad f(\infty) = 0, \quad (9)$$

with R the bubble radius and σ the wall-steepness parameter. The analytic derivative used in the Natário formulas is

$$f'(r) = \sigma \frac{\text{sech}^2 \sigma(r + R) - \text{sech}^2 \sigma(r - R)}{2 \tanh(\sigma R)}. \quad (10)$$

Code: warpbubblesim/metrics/alcubierre.py, natario.py, base.py::tanh_shape.

2 Geodesics: how light travels through the bubble

A photon’s worldline in any spacetime is a null geodesic. In coordinate basis the geodesic equation is

$$\frac{d^2 x^\mu}{d\lambda^2} + \Gamma_{\alpha\beta}^\mu \frac{dx^\alpha}{d\lambda} \frac{dx^\beta}{d\lambda} = 0, \quad (11)$$

with λ an affine parameter and the Christoffel symbols

$$\Gamma_{\alpha\beta}^\mu = \frac{1}{2} g^{\mu\nu} (\partial_\alpha g_{\beta\nu} + \partial_\beta g_{\alpha\nu} - \partial_\nu g_{\alpha\beta}). \quad (12)$$

Defining the tangent $k^\mu \equiv dx^\mu/d\lambda$, the *null* condition is $g_{\mu\nu} k^\mu k^\nu = 0$, conserved along the geodesic by (11).

We rewrite (11) as the first-order system

$$\boxed{\dot{x}^\mu = k^\mu, \quad \dot{k}^\mu = -\Gamma_{\alpha\beta}^\mu k^\alpha k^\beta,} \quad (13)$$

and integrate with classical Runge–Kutta. The renderer’s high-throughput JAX path uses fixed-step **RK4** inside `jax.lax.scan` so the entire trajectory unrolls into a single XLA kernel that vectorises over all pixels.

2.1 Computing $\partial_\alpha g_{\mu\nu}$

Two backends are supported.

Forward-mode autodiff. `jax.jacfwd(metric)(coords)` returns the Jacobian $\partial_\alpha g_{\mu\nu}$ exactly to machine precision, with no finite-difference noise. This is the default for Alcubierre.

Vectorised central differences. For Natário the curl-based shift (??) contains $g'(r_s)/r_s$ factors. The function value is regular at $r_s = 0$ (the limit equals $-2\sigma^2 \text{sech}^2(\sigma R)$), but the *Jacobian* is a 0/0 that produces NaNs under `jacfwd`. We fall back to

$$(\partial_\alpha g_{\mu\nu})(\mathbf{x}) \approx \frac{g_{\mu\nu}(\mathbf{x} + h \mathbf{e}_\alpha) - g_{\mu\nu}(\mathbf{x} - h \mathbf{e}_\alpha)}{2h}, \quad h \sim 10^{-3}, \quad (14)$$

vectorised over the four basis directions with `jax.vmap`.

Code: `warpbubblesim/viz/skyrender_jax.py::make_geodesic_rhs`; `warpbubblesim/gr/tensors.py` (CPU reference).

3 The observer’s frame: orthonormal tetrads

A pinhole camera lives in the observer’s local rest frame, which is an orthonormal tetrad $\{e_{\hat{a}}^\mu\}_{a=0}^3$ such that

$$g_{\mu\nu} e_{\hat{a}}^\mu e_{\hat{b}}^\nu = \eta_{\hat{a}\hat{b}} = \text{diag}(-1, +1, +1, +1). \quad (15)$$

We always take $e_{\hat{0}} \equiv u$ (the observer’s 4-velocity) and build the spatial legs by Gram–Schmidt against the spacetime metric. Given two coordinate-basis seeds $\hat{\mathbf{f}}$ (forward, default ∂_x) and $\hat{\mathbf{u}}p$ (up, default ∂_z), the renderer constructs

$$e_{\hat{0}} = \frac{u}{\sqrt{-g(u, u)}}, \quad (16)$$

$$\tilde{e}_{\hat{1}} = \hat{\mathbf{f}} - g(\hat{\mathbf{f}}, e_{\hat{0}}) e_{\hat{0}}, \quad e_{\hat{1}} = \tilde{e}_{\hat{1}} / \sqrt{g(\tilde{e}_{\hat{1}}, \tilde{e}_{\hat{1}})}, \quad (17)$$

$$\tilde{e}_{\hat{3}} = \hat{\mathbf{u}}p - \sum_{a<3} g(\hat{\mathbf{u}}p, e_{\hat{a}}) e_{\hat{a}}, \quad e_{\hat{3}} = \tilde{e}_{\hat{3}} / \sqrt{g(\tilde{e}_{\hat{3}}, \tilde{e}_{\hat{3}})}, \quad (18)$$

$$e_{\hat{2}} = \text{Gram–Schmidt of } \partial_y \text{ against } \{e_{\hat{0}}, e_{\hat{1}}, e_{\hat{3}}\}, \quad (19)$$

with a final right-handedness flip on $e_{\hat{2}}$ if the spatial 3-determinant is negative.

Bubble-centre observer

The natural observer co-moves with the bubble centre. For Alcubierre at $r_s = 0$ this is $u = (1, v_s, 0, 0)$, exactly normalised by the centre metric. In the renderer this u is checked against $g(u, u) < 0$ before proceeding; if not, a fallback to the static seed $u = (1, 0, 0, 0)$ is tried, and only if that also fails do we abort with “no timelike observer.” Natário rejects this fallback for $v_s \geq 1$ (its central worldline is spacelike there), which is why the comparison sweep is sub-luminal.

Code: `warpbubblesim/viz/skyrender.py::build_orthonormal_tetrad`.

4 Camera model: pixels to past-pointing null tangents

A pinhole camera with horizontal field of view Φ , image dimensions $W \times H$ pixels, and aspect $A = W/H$ produces, for pixel (i, j) :

$$u_{ij} = \left(\frac{2(i+0.5)}{W} - 1 \right) \tan(\Phi/2), \quad (20)$$

$$v_{ij} = \left(1 - \frac{2(j+0.5)}{H} \right) \frac{\tan(\Phi/2)}{A}. \quad (21)$$

The camera’s local-frame ray direction (along $+\hat{x}$ by default) is

$$\hat{n}_{ij} = \frac{(1, u_{ij}, v_{ij})}{\sqrt{1 + u_{ij}^2 + v_{ij}^2}} \in S^2 \subset \mathbb{R}^3. \quad (22)$$

A photon *seen by* the observer arriving from direction $\hat{n}$ has future-pointing 4-momentum $(\omega, -\omega\hat{n})$ in the local frame (it travels *toward* the camera in direction $-\hat{n}$). Reversing time gives the past-pointing tangent $(-\omega, \omega\hat{n})$, and an affine-reparameterisation removes ω . In the coordinate basis this is

$$k_{\text{init}}^\mu = -e_{\hat{0}}^\mu + \hat{n}^i e_i^\mu. \quad (23)$$

Anti-aliasing. For supersample factor S , each pixel contributes S^2 rays at deterministically jittered sub-pixel offsets; the final pixel colour is the mean over the S^2 samples after sky lookup. Default $S = 2$ (4 rays/pixel) is enough to suppress sub-pixel aberration flicker for animations.

Code: `warpbubblesim/viz/skyrender.py::Camera`; `skyrender_jax.py::pixel_directions_np`.

5 Mapping the asymptotic direction to the celestial sphere

The integration (13) is run with initial state $(x_{\text{init}}^\mu, k_{\text{init}}^\mu) = (x_{\text{obs}}, k_{\text{init}})$ for $N_{\text{steps}} \Delta\lambda$ in affine parameter — long enough for the ray to leave the bubble’s sphere of influence (typically $r_s \gtrsim 5R$). After that the metric is asymptotically Minkowski, so k^μ is parallel-transported = constant in coordinates, and the spatial part of the final tangent gives the lab-frame direction the photon came from:

$$\hat{n}_{\text{asy}} = \frac{(k_{\text{final}}^x, k_{\text{final}}^y, k_{\text{final}}^z)}{|(k_{\text{final}}^x, k_{\text{final}}^y, k_{\text{final}}^z)|}. \quad (24)$$

We then sample a celestial-sphere texture $T : S^2 \rightarrow [0, 1]^3$ at $\hat{n}_{\text{asy}}$. Two backends are provided:

- **Procedural starfield.** Stars sampled uniformly on S^2 with brightness drawn from a heavy-tailed distribution; a soft top- k Gaussian blend gives anti-aliased points.
- **Equirectangular panorama.** (θ, ϕ) from $\hat{n}_{\text{asy}}$ via $\theta = \arccos \hat{n}_z$, $\phi = \text{atan2}(\hat{n}_y, \hat{n}_x)$; bilinear interpolation in pixel-space with periodic wrap in longitude.

Code: `warpbubblesim/viz/skybackground.py`.

6 Honest physics: Doppler brightness and front horizon

6.1 Doppler factor from Killing energy

The energy of a photon with 4-momentum p^μ measured by an observer with 4-velocity u^μ is $\omega = -g_{\mu\nu}p^\mu u^\nu$. Our *past-pointing* tangent satisfies $k = -p$, so $\omega = g_{\mu\nu}k^\mu u^\nu$ is positive. For a backward null ray that leaves the bubble and reaches the asymptotic Minkowski region, the source’s static observer is $u_{\text{src}} = (1, 0, 0, 0)$ in the lab frame, evaluated against $\eta_{\mu\nu}$, and the Doppler factor is

$$f = \frac{\omega_{\text{obs}}}{\omega_{\text{src}}} = \frac{g_{\mu\nu}(\mathbf{x}_{\text{obs}}) k_{\text{init}}^\mu u_{\text{obs}}^\nu}{\eta_{\mu\nu} k_{\text{final}}^\mu u_{\text{src}}^\nu}. \quad (25)$$

$f > 1$ is blueshift, $f < 1$ is redshift.

6.2 Brightness scaling: Liouville’s invariant

Liouville’s theorem applied to the photon distribution function in vacuum gives the conserved invariant

$$\frac{I_\nu}{\nu^3} = \text{const along the null geodesic}, \quad (26)$$

so monochromatic specific intensity transforms as $I_\nu^{\text{obs}} = f^3 I_\nu^{\text{src}}$. For a thermal (blackbody) source integrated over frequency, bolometric intensity goes as f^4 . Both are physically defensible — different spectrum assumptions — and the renderer exposes the exponent as `doppler_intensity_power`. No colour shift is applied because the input panorama is single-band visible-light RGB and any “wavelength rotation” would be invented.

6.3 Front horizon as a black region

For $|v_s| > 1$ in Alcubierre, there is a region of the celestial sphere from which no light can reach the bubble interior — sources directly in front have already been overtaken by the bubble at the time their light would arrive. Operationally, in the renderer:

$$\text{ray } i \text{ trapped} \iff r_s(\mathbf{x}_{\text{final}}^{(i)}) < \kappa R, \quad (27)$$

with default safety factor $\kappa = 1.5$. Trapped pixels are rendered black. No Hawking glow is invented; computing the actual horizon temperature requires the surface gravity κ_H from the metric profile, which the renderer doesn’t extract.

Code: `warpbubblesim/viz/effects.py`.

7 Numerical scheme summary

1. **Build observer state.** Compute $\mathbf{x}_{\text{obs}} = (t, x_s(t), 0, 0)$, $u = (1, v_s, 0, 0)$ (after fallback check), tetrad $e_{\hat{a}}$.
2. **Build initial null tangents.** For each pixel sub-sample $\hat{n}_{ij}^{(s)}$, set $k_{\text{init}}^\mu = -e_{\hat{0}} + \hat{n}^{\hat{i}} e_{\hat{i}}$.
3. **Integrate.** RK4 on (13) with step $\Delta\lambda$ for N_{steps} steps, vectorised across all rays. In JAX the integration is jit-compiled and vmap’d; in NumPy the metric Jacobian is built per RK stage and broadcast over the pixel axis.
4. **Asymptotic direction.** Read $\hat{n}_{\text{asy}}$ from the spatial part of k_{final} .
5. **Sky lookup.** Sample $T(\hat{n}_{\text{asy}})$.
6. **Doppler.** Multiply by f^p from (25).

7. **Horizon mask.** Replace pixels with $r_s^{\text{final}} < \kappa R$ by black, only when $|v_s| > 1$.

8. **Anti-alias.** Mean of S^2 sub-samples per pixel.

Cost scaling. A frame at $W \times H$ pixels with supersample S and integration budget N_{steps} requires

$$N_{\text{rays}} N_{\text{steps}} N_{\text{stages}} N_{\text{metric calls per Christoffel}} \quad (28)$$

metric evaluations, with $N_{\text{rays}} = WHS^2$, $N_{\text{stages}} = 4$ (RK4), and $N_{\text{metric calls}} = 1$ for autodiff or $= 9$ for FD-Christoffels. At HQ (1280×720 , $S = 2$, $N_{\text{steps}} = 280$): $\sim 10^{10}$ metric evaluations per frame — easily a single-GPU minute.

8 Velocity schedule

The animated “flight” is a sequence of independent steady-state renders at velocities $v_s^{(i)}$ for frame index $i \in \{0, \dots, N-1\}$. The default schedule is the half-cosine ease-in-out

$$v_s^{(i)} = V_{\text{max}} \frac{1 - \cos(\pi \tau_i)}{2}, \quad \tau_i = \frac{i}{N-1}, \quad (29)$$

which has zero acceleration at the endpoints and peak acceleration at the midpoint. The assembled video is $[v = 0 \rightarrow V_{\text{max}} \rightarrow 0]$ via [forward frames || 6 holds || reversed frames].

9 Conventions cheatsheet

Symbol / object	Meaning / convention
$g_{\mu\nu}$, signature	4-metric, $(-, +, +, +)$
x^μ , $\mu = 0..3$	(t, x, y, z)
$\Gamma_{\alpha\beta}^\mu$	Christoffel of the second kind, eq. (12)
$k^\mu = dx^\mu/d\lambda$	null tangent, $g_{\mu\nu} k^\mu k^\nu = 0$
k in the renderer	past-pointing ($k^t < 0$)
$\omega = g_{\mu\nu} k^\mu u^\nu$	observed photon energy (positive for past-pointing k)
$f = \omega_{\text{obs}}/\omega_{\text{src}}$	Doppler factor
f^3 vs f^4	Liouville monochromatic vs. bolometric
“forward axis”	$+\hat{x}$ in the camera’s local rest frame
horizon mask	$r_s^{\text{final}} < 1.5 R$, only for $ v_s > 1$

10 A common misconception

A natural objection — repeated in many casual write-ups — is that “crossing c ” should imply an extreme blueshift via the SR formula $f_{\text{SR}} = \sqrt{(1+v)/(1-v)}$, so the visible band would shift into hard X-rays / gamma rays and the cabin should “see nothing”. **This is incorrect for the Alcubierre interior observer.**

The SR Doppler formula governs a Lorentz-boosted inertial observer in flat space, where a relative velocity v between source and receiver is meaningful and bounded by c . The interior observer is, by construction, *not* that observer — the warp metric makes them locally at rest with respect to the bubble; the surrounding spacetime is doing the moving. Concretely, on the bubble-centre worldline $x = x_s(t)$ at velocity v_s :

$$g_{\mu\nu} u^\mu u^\nu = -1 \quad \text{for any } v_s, \quad (30)$$

so the observer’s local rest frame is well-defined and the photon energy in that frame is the standard tetrad projection $\omega_{\text{obs}} = g_{\mu\nu} k^\mu u^\nu$. For an on-axis forward-looking ray, the null condition

gives $k_{\text{init}}^x = 1 - v$ in coordinates; the Killing energy at the centre is $-k_t = v - 1$. After integration through the warp wall the asymptotic frequency in the lab frame is finite, and the resulting Doppler factor

$$f = \omega_{\text{obs}}/\omega_{\text{src}} \quad (31)$$

takes **single-digit values** at multi-c bubble velocities in practice (numerical traces give $f \approx 3.45$ at $v_s = 2.5c$ in our test geometry; the trend with v_s is mild, not divergent). The visible band shifts to soft UV at the brightest forward pixels — some signal is lost there — but *the renderer’s f^3 scaling applied to a visible-band panorama still yields a clearly recognisable saturating forward bloom*. “No light at all” would require $f \sim 10^5$ – 10^6 to push 500 nm into the gamma-ray band, which the Alcubierre central-observer geometry simply does not produce.

The genuinely non-trivial superluminal-warp effects are elsewhere:

- the front horizon (rendered as black via the horizon mask),
- the matter pile-up at the leading bubble wall (McMonigal et al. 2012; not visible to the interior crew during cruise — outside the front horizon),
- Hawking-like quantum particle creation at the front horizon (Finazzi–Liberati–Barceló 2009; predicted to destabilise the bubble, not modelled here).

See also

- `notebooks/07_warp_skyflight_colab.ipynb` — Drive-checkpointed Colab GPU notebook.
- `Wolfram/skyrender/SkyRender.wl` — Mathematica implementation, optionally offloaded to a Wolfram Cloud GPU runtime via `RemoteBatchSubmit`.